

ShieldNet AI: Customer Solution Brief

Version: April 2026

1. Executive Summary

ShieldNet AI is a real-time malicious URL detection platform that combines deterministic analysis, semantic vector retrieval, Atlas text search, and optional Foundry-based agentic reasoning.

It helps security teams triage suspicious URLs faster with explainable evidence and campaign-level context.

2. Business Value

- Accelerates URL triage through automated risk scoring and evidence cards.
- Improves detection quality using lexical + semantic retrieval.
- Supports campaign intelligence by clustering related malicious URLs.
- Enables analyst workflows using threat database filters and status operations.

3. High-Level Architecture

Frontend: Next.js 15 + React 18 + Recharts

Backend: FastAPI + Motor + Pydantic

Data & Search: MongoDB Atlas (Atlas Search + Vector Search + Hybrid Rank Fusion)

Embeddings: Voyage API (voyage-4 default, 1024 dimensions)

Optional Agentic Layer: Microsoft Foundry (gpt-5.4 default)

4. Key Functional Modules

- URL Scanner: extraction, scoring, vector evidence, recommendation.
- Threat Database: searchable repository with operational triage.
- Search Playground: Atlas vs Vector vs Hybrid comparison.
- Campaign Detection: groups related threats and tags URLs.
- Rules Management: active threshold and weight configuration.
- URL Graph: relation edges between similar threats.

4.1 Backend Module Stack

API layer

FastAPI + Uvicorn + Pydantic: defines REST endpoints, request validation, response typing, and ASGI runtime.

Data layer

Motor (async MongoDB driver): repository-pattern CRUD, aggregations, and index-aware querying.

Config layer

python-dotenv + environment variables: loads runtime secrets and deployment configuration values.

HTTP integration

httpx: calls Voyage embedding API, Foundry LLM endpoint, and external enrichment APIs.

URL scoring logic

Custom risk engine + python-Levenshtein: threat scoring, typo-tolerant phishing keyword matching, and classification floors/thresholds.

Enrichment DNS

dnspython: A/AAAA/CNAME/MX/NS/TXT lookups, DNSSEC checks, and reverse DNS enrichment.

Enrichment WHOIS

python-whois: domain age, registrar profile, privacy hints, and expiry-window signals.

Enrichment TLS

ssl + socket: certificate type, issuer confidence, domain-match checks, recency, and expiry risk signals.

Testing

pytest + pytest-asyncio: unit tests and async behavior validation across backend modules.

5. Data Sources and Ingestion

- Live scan ingestion writes/updates records in urls collection.
- Threat-intel seed writes threat docs into urls using docType=threat_intel.
- Campaign detection writes campaign aggregates in campaigns.
- Similarity graph writes edges into url_edges.
- Active policy config is maintained in scan_rules.

6. Sample Schemas

6.1 urls (scan record)

```
{
  "url": "https://secure-login-incometax.xyz/auth/verify",
  "domain": "secure-login-incometax.xyz",
  "canonicalDomain": "incometax.gov.in",
  "docType": "scan",
  "threatClassification": "phishing",
  "riskScore": 86.4,
  "status": "blocked",
  "summaryText": "Credential harvest style URL impersonating tax portal...",
  "embedding": ["1024-dim vector"],
  "payloadTypes": ["credential_harvest", "open_redirect"],
  "campaignId": "campaign_001_tax_auth_2025",
  "scanCount": 3,
  "firstSeenAt": "2026-04-27T08:51:00Z",
  "lastSeenAt": "2026-04-30T11:08:12Z"
}
```

6.2 urls (threat-intel record in same collection)

```
{
  "url": "https://nic.in/login?redirect=http://gov-login-verify.xyz/capture",
  "domain": "nic.in",
  "docType": "threat_intel",
  "threatClassification": "phishing",
  "attackCategory": "open_redirect",
  "feedName": "CERT-IN_Feed",
  "summaryText": "Open redirect attack against nic.in ...",
  "embedding": ["1024-dim vector"]
}
```

6.3 campaigns

```
{
```

```

    "campaignId": "campaign_001_tax_auth_2025",
    "name": "Tax Authority Phishing Campaign Q1 2025",
    "attackCategory": "credential_harvest",
    "status": "active",
    "urlCount": 128,
    "avgRiskScore": 83.7,
    "domains": ["incometax.gov.in", "gst.gov.in"],
    "lastSeen": "2026-04-30T10:21:00Z"
  }
}

6.4 url_edges
{
  "fromUrl": "https://secure-login-incometax.xyz/auth/verify",
  "toUrl": "https://gst-auth-update.top/kyc/verify",
  "strength": 0.79,
  "factors": [
    {"name": "vector_similarity", "value": 0.84},
    {"name": "shared_attack_category", "value": 0.74}
  ]
}

6.5 scan_rules
{
  "_id": "active_rules",
  "thresholds": {"block": 70, "review": 45},
  "weights": {"vector": 0.30, "payload": 0.20, "dga": 0.20}
}

```

7. Embedding Strategy

- Embed summaryText (URL + metadata + threat signals), not raw URL alone.
- Default model: voyage-4; vector size: 1024; similarity: cosine.
- Campaign-aware re-embedding improves future threat/campaign retrieval precision.

8. Detection Flow

1) Input URL -> 2) Feature extraction -> 3) Summary + embedding -> 4) Vector search -> 5) Risk engine
-> 6) Campaign tag/update -> 7) Persist + response

9. URL Scoring System (Detailed)

ShieldNet computes a composite risk score from 0 to 100. The score blends deterministic checks, semantic similarity evidence, visual impersonation signals, and analyst feedback so each verdict is evidence-backed and explainable.

9.1 Scoring bands

- 0-44: low risk (allow/monitor).
- 45-69: medium risk (analyst review queue).
- 70-100: high risk (block/quarantine recommendation).

9.2 Weighted signal composition

- Visual impersonation checks: look-alike branding, suspicious login flows, and favicon/title mismatch indicators.
- Vector evidence: cosine similarity against known phishing and campaign exemplars in Atlas Vector Search.
- Lexical/search evidence: Atlas text search matches for payload patterns, abuse terms, and known lure phrases.
- Structural and DNS/TLS checks: redirect depth, domain age/entropy, resolver anomalies, and certificate posture.

- Human feedback: analyst-confirmed true/false positives used to adjust thresholds/weights and improve precision over time.

9.3 Explainability

Each scan returns a score plus supporting factors so analysts can see why a URL was classified as block, review, or allow. This improves auditability and speeds triage decisions.

9.4 How scoring works in practice

- Step A: normalize and parse URL structure (scheme, host, path, query, redirects).
- Step B: lexical and typo analysis with Levenshtein-style distance against phishing terms and protected brands.
- Step C: enrichment evidence (DNS/WHOIS/TLS) contributes positive or negative risk deltas.
- Step D: semantic retrieval computes vector similarity to prior malicious URLs/campaign exemplars.
- Step E: weighted fusion computes base score, then classification floor rules prevent under-scoring for high-confidence phishing patterns.
- Step F: policy thresholds map score to allow/review/block with reason codes for analyst traceability.

10. Pipeline Mode vs Foundry Mode

ShieldNet supports two execution modes so customers can balance cost, speed, and depth of reasoning.

10.1 Deterministic pipeline mode (default)

- FastAPI pipeline runs feature extraction, vector/text retrieval, rule scoring, and policy mapping.
- Best for high-throughput screening, lower latency, and predictable cost.
- Produces stable risk outputs suitable for SIEM/SOAR ingestion.

10.2 Foundry agentic mode (optional)

- Adds Microsoft Foundry reasoning to enrich narrative, summarize evidence, and propose analyst actions.
- Useful for complex or ambiguous cases where additional context synthesis improves analyst confidence.
- Can run as a selective escalation step only for medium-confidence URLs.

10.3 Practical operating model

Common deployment: process all URLs through deterministic pipeline first, then route only borderline or high-impact cases to Foundry for deeper explanation and response guidance.

11. How Visual Checks + Vector Search + Human Feedback Improve URL Scoring

- Visual checks capture brand impersonation and deceptive UX indicators that raw text features may miss.
- Vector search finds semantically similar malicious behavior even when attackers mutate tokens/domains.
- Human feedback closes the loop by correcting edge cases and reducing recurring false positives.
- Combined effect: stronger recall on novel attacks while preserving precision on legitimate look-alike traffic.

This blended approach is especially effective against phishing campaigns that evolve quickly across domains, paths, and social-engineering language.

12. Hybrid Enforcement Engine Stages

The Hybrid Enforcement Engine combines deterministic controls with semantic retrieval and optional agentic escalation.

- Stage 1: Intake and normalization - sanitize URL input, extract canonical features, and build analysis context.
- Stage 2: Deterministic risk checks - apply rule-based signals (payload patterns, DGA/entropy, redirects, DNS/TLS anomalies).
- Stage 3: Hybrid retrieval - run Atlas text search and vector search, then apply rank fusion for stronger evidence coverage.
- Stage 4: Composite scoring - merge deterministic + retrieval + visual impersonation signals into a single risk score.
- Stage 5: Enforcement decision - map score and confidence to allow, analyst-review, or block actions.
- Stage 6: Analyst feedback loop - capture dispositions to tune thresholds/weights and improve future precision/recall.
- Stage 7: Optional Foundry escalation - invoke Foundry reasoning for ambiguous/high-impact cases to enrich explanation and response guidance.

13. Operational Notes

- This is a validation/demo deployment and should be calibrated for production traffic.
- Keep secrets in environment variables and enforce role-based access on rules/status overrides.

Prepared for customer sharing. Customize with project branding, SLAs, and rollout plan.